

Multi-Objective Optimization

Other Advanced Multi-Objective Metaheuristics: Core Ideas and Mechanisms

Bruno de Athayde Prata

Lecture Overview

Advanced Non-GA Classes of Multi-Objective Metaheuristics

1 Introduction & Taxonomy

- Beyond Genetic Algorithms
- Local search, trajectory methods, and swarm intelligence

2 MOTS — Multi-Objective Tabu Search (Gandibleux et al., 1997)

3 PSA — Pareto Simulated Annealing (Czyzak & Jaskiewicz, 1998)

4 MOPSO — Multi-Objective Particle Swarm Optimization (Coello Coello & Lechuga, 2002; Banks et al., 2008)

5 2PPLS — Two-Phase Pareto Local Search (Lust & Teghem, 2010)

6 MO-GRASP — Multi-Objective GRASP with Path Relinking (Martí et al., 2015)

7 Comparative Framework and Key Takeaways

Motivation & Core Concept:

- Extend deterministic Tabu Search memory mechanisms to multi-objective combinatorial problems.
- Avoid local traps by tracking recently visited moves in a **Tabu List** (TL_move).
- Evaluate neighborhood candidates $V(x)$ using a **scalarizing regret function** relative to an ideal/utopian point $\tilde{z}$:

$$S(z, q, \omega) = \|z - \tilde{z}\|_{q, \omega} = \left[\sum_{j=1}^p w_j^q |z_j - \tilde{z}_j|^q \right]^{1/q}.$$

Key Search Mechanisms:

- **Compromise Selection:** Select a non-tabu neighbor $\hat{x}$ minimizing $S(f(x), q, \omega)$.
- **Diversification via Weight Update:** Dynamically adjust the weight vector ω based on performance steps $(t_{\min}, t_{\max})$.
- **Objective Tabu List:** Place objectives in TL_objective to prevent over-focusing on recently improved criteria.

Algorithm 1 Multi-Objective Tabu Search (Gandibleux et al., 1997)

Require: Initial $x_0 \in D$, ω , $G_{\max}$, steplter

```

1: Set  $\hat{E}(P) \leftarrow \{x_0\}$ 
2:  $\text{TL\_move} \leftarrow \emptyset$ 
3:  $\text{TL\_objective} \leftarrow \emptyset$ 
4:  $k \leftarrow 0$ 
5: while  $k < G_{\max}$  do
6:   for  $s = 1$  to steplter do
7:     Compute neighborhood  $V(x_k)$  and ideal point
        
$$\tilde{z}_j = \max_{x' \in V(x_k)} f_j(x')$$

8:     Select best admissible candidate  $\hat{x} \in V(x_k) \setminus \text{TL\_move}$  minimizing  $S(f(x'), q, \omega)$ 
9:     Update  $\text{TL\_move}$  with move  $x_k \rightarrow \hat{x}$ 
10:    Update non-dominated set  $\hat{E}(P)$  with  $\hat{x}$ 
11:    Check aspiration criterion for forbidden  $x^\# \in \text{TL\_move}$ 
12:     $k \leftarrow k + 1$ 
13:     $x_k \leftarrow \hat{x}$ 
14:  end for
15:  procedure UPDATE_WEIGHTS ( $\omega, \text{TL\_objective}$ )
    Decrease  $w_j$  for significantly improved objectives
    Increase  $w_j$  for unimproved non-tabu objectives
16: end while
17: return approximation set  $\hat{E}(P)$ 
    
```

Core Idea:

- Extends classical Simulated Annealing by maintaining a **population of generating solutions** S .
- Combines probabilistic move acceptance with an adaptive weighting vector $\Lambda = [\lambda_1, \dots, \lambda_J]$.

Interactive Weight Tuning (Repulsion Mechanism):

- For solution $x \in S$, locate its closest non-dominated neighbor $x' \in S$.
- Modify weights λ_j^x to increase the distance between x and x' :

$$\lambda_j^x \leftarrow \begin{cases} \alpha \lambda_j^x, & \text{if } f_j(x) \geq f_j(x'), \\ \lambda_j^x / \alpha, & \text{if } f_j(x) < f_j(x'). \end{cases} \quad (\alpha > 1).$$

- Normalize the weights so that $\sum_j \lambda_j^x = 1$.
- This increases the probability of accepting moves away from x' .

Algorithm 2 Pareto Simulated Annealing (Czyzak & Jaskiewicz, 1998)**Require:** Initial sample $S \subset D$, initial temperature T_0 , cooling rate $\gamma \in (0, 1)$, $\alpha > 1$

```

1: Initialize archive  $M \leftarrow \emptyset$ 
2: Update  $M$  with all  $x \in S$ 
3:  $T \leftarrow T_0$ 
4: repeat
5:   for each  $x \in S$  do
6:     Construct candidate neighbor  $y \in V(x)$ 
7:     if  $y$  is not dominated by  $x$  then
8:       Update non-dominated archive  $M$  with  $y$ 
9:     end if
10:    Find closest solution  $x' \in S \setminus \{x\}$  non-dominated w.r.t.  $x$ 
11:    if  $x'$  exists then
12:      for each objective  $j = 1, \dots, J$  do
13:         $\lambda_j \leftarrow \alpha \lambda_j$  if  $f_j(x) \geq f_j(x')$ 
14:         $\lambda_j \leftarrow \lambda_j / \alpha$  otherwise
15:      end for
16:      Normalize  $\Lambda$  such that  $\sum_{j=1}^J \lambda_j = 1$ 
17:    else
18:      Assign random non-negative normalized weight vector  $\Lambda$ 
19:    end if
20:    Compute acceptance probability:

```

$$P(x, y, T, \Lambda) = \min \left(1, \exp \left(\sum_{j=1}^J \lambda_j \frac{f_j(y) - f_j(x)}{T} \right) \right)$$

```

21:    Accept  $x \leftarrow y$  with probability  $P(x, y, T, \Lambda)$ 
22:  end for
23:  Decrease temperature  $T \leftarrow \gamma T$ 
24: until stopping condition is met
25: return potentially efficient set  $M$ 

```

Core Concepts:

- Adapts velocity and position updates of PSO to multi-objective domains using a **repository (archive)** of non-dominated solutions.
- Uses an adaptive **hyper-grid** in objective space to preserve distribution uniformness.

Velocity & Leader Selection Strategy:

- Velocity update equation:

$$v_{i,t+1} = \omega v_{i,t} + c_1 r_1 (p_i - x_{i,t}) + c_2 r_2 (g_i - x_{i,t}).$$

- p_i is the personal best and g_i is a leader selected from the external repository.
- **Leader Selection (g_i):** Hyper-boxes containing fewer particles receive higher selection probabilities.
- **Mutation Operator:** Applies turbulence/mutation to particle positions to prevent premature convergence.

Algorithm 3 Multi-Objective Particle Swarm Optimization

Require: Swarm size N , maximum generations $G_{\max}$, repository size $R_{\max}$

```

1: Initialize population  $P(0)$  with random positions and zero velocities
2: Evaluate objectives for all particles in  $P(0)$ 
3: Store non-dominated particles of  $P(0)$  in external repository  $R$ 
4: Grid target objective space and assign grid location to archive members
5: Set personal best  $p_i \leftarrow x_i(0)$  for all  $i = 1, \dots, N$ 
6:  $t \leftarrow 0$ 
7: while  $t < G_{\max}$  do
8:   for each particle  $i = 1, \dots, N$  do
9:     Select leader  $g_i \in R$  using roulette-wheel selection based on grid crowding
10:
11:     
$$v_i(t+1) \leftarrow \omega v_i(t) + c_1 r_1 (p_i - x_i(t)) + c_2 r_2 (g_i - x_i(t))$$

12:      $x_i(t+1) \leftarrow x_i(t) + v_i(t+1)$ 
13:     Apply mutation/turbulence operator
14:     Evaluate objective functions  $f(x_i(t+1))$ 
15:     Update personal best  $p_i$  if  $x_i(t+1)$  dominates  $p_i$ 
16:   end for
17:   Update repository  $R$  with new non-dominated solutions
18:   Prune  $R$  using hyper-grid if  $|R| > R_{\max}$ 
19:    $t \leftarrow t + 1$ 
20: end while
return external repository  $R$ 
    
```

Framework Overview:

- Parameter-free pure local search metaheuristic combining scalarized initialization with Pareto local exploration.

Phase 1: Supported Efficient Solutions Approximation

- Applies Aneja & Nair's dichotomic scheme with single-objective solvers.
- Generates a rich initial population $\hat{X}_{SE1_m}$ representing extreme supported efficient solutions.

Phase 2: Pareto Local Search (PLS)

- Explores the neighborhood of every solution in the population.
- Adds non-weakly dominated neighbors to the active population.
- Terminates when no further non-dominated neighbors can be found.

Algorithm 4 Two-Phase Pareto Local Search**Require:** Multi-objective instance matrices C_1, C_2 , single-objective solver SOLVESO**Phase 1: Initial Population Generation**

- 1: Solve lexicographic optimal solutions x_1, x_2 using SOLVESO
- 2: $\hat{X}_E \leftarrow \{x_1, x_2\}$
- 3: Push pair (x_1, x_2) onto stack Q
- 4: **while** $Q \neq \emptyset$ **do**
- 5: Pop (x_r, x_s) from Q
- 6: Compute normal weights:

$$\lambda_1 = \frac{z_2(x_r) - z_2(x_s)}{[z_2(x_r) - z_2(x_s)] + [z_1(x_s) - z_1(x_r)]}, \quad \lambda_2 = 1 - \lambda_1$$

- 7: $x_t \leftarrow \text{SOLVESO}(\lambda_1 C_1 + \lambda_2 C_2)$
- 8: **if** $z(x_t)$ lies strictly inside $\Delta z(x_r)z(x_s)$ **then**
- 9: Update $\hat{X}_E$ with x_t
- 10: Push (x_r, x_t) and (x_t, x_s) onto Q

- 11: **end if**

- 12: **end while**

Phase 2: Pareto Local Search

- 13: $P \leftarrow \hat{X}_E$
- 14: **while** $P \neq \emptyset$ **do**
- 15: Select $p \in P$
- 16: $P \leftarrow P \setminus \{p\}$
- 17: **for** each neighbor $p' \in \mathcal{N}(p)$ **do**
- 18: **if** p' is not weakly dominated by any member of $\hat{X}_E$ **then**
- 19: Add p' to $\hat{X}_E$
- 20: Remove dominated solutions
- 21: Add p' to active population P
- 22: **end if**
- 23: **end for**
- 24: **end while**
- 25: **return** approximation set $\hat{X}_E$

MO-GRASP with Path Relinking

Multi-Objective GRASP + PR (Martí et al., 2015)

Construction Strategies:

- **Pure:** Single objective guides construction.
- **Sequential Combined:** Switches objectives at each construction step.
- **Weighted Combined:** Combines objectives through $g(c) = \sum w_j g_j(c)$.

Local Search Strategies:

- **Pure LS:** Improves a single objective.
- **Sequential LS:** Rotates objectives and accepts non-dominating moves.
- **Weighted LS:** Operates on $f(x) = \sum w_j f_j(x)$.

Path Relinking Integration:

- Connects elite solutions in the non-dominated set ES by swapping solution attributes.
- Collects non-dominated intermediate path solutions into an archive.

MO-GRASP

Algorithm Pseudocode

Algorithm 5 Multi-Objective GRASP with Path Relinking

Require: Stopping condition, elite set size limit, strategy (Pure / Sequential / Weighted)

```
1:  $ES \leftarrow \emptyset$ 
2: while GRASP stopping criterion not satisfied do
3:    $x \leftarrow \emptyset$ 
4:   while construction not complete do
5:     Evaluate candidate set  $C$  using chosen strategy
6:     Build restricted candidate list  $RCL$ 
7:     Select candidate  $c^* \in RCL$  at random
8:      $x \leftarrow x \cup \{c^*\}$ 
9:   end while
10:   $x \leftarrow \text{MULTIOBJECTIVELOCALSEARCH}(x)$ 
11:  Update elite set  $ES$  with  $x$  if non-dominated
12: end while
Post-Processing: Multi-Objective Path Relinking
13:  $IP \leftarrow \emptyset$ 
14: for all pairs  $(x, y) \in ES \times ES, x \neq y$  do
15:    $p_0 \leftarrow x$ 
16:    $El_{x-y} \leftarrow x \setminus y$ 
17:    $El_{y-x} \leftarrow y \setminus x$ 
18:   while  $El_{x-y} \neq \emptyset$  do
19:     Find move replacing  $i \in El_{x-y}$  with  $j \in El_{y-x}$ 
20:     Apply swap to yield intermediate solution  $p_k$ 
21:      $IP \leftarrow IP \cup \{p_k\}$ 
22:   end while
23: end for
24:  $ES \leftarrow \text{NonDominatedFilter}(ES \cup IP)$ 
25: return Pareto approximation  $ES$ 
```

▷ Intermediate Pool

Comparison of Advanced Metaheuristic Classes

Summary Table

Algorithm	Search Framework	Diversity Mechanism	Archive / Elitism
MOTS	Tabu Search	Dynamic weights + Objective Tabu List	Updated set $\hat{E}(P)$
PSA	Simulated Annealing	Neighbor-repulsion weight tuning	External archive M
MOPSO	Particle Swarm	Adaptive objective-space hyper-grid	External repository R
2PPLS	Dichotomic PLS +	Phase 1 extreme points + neighborhood exploration	Active population P + set $\hat{X}_E$
MO-GRASP	GRASP + Path Relinking	Randomized RCL + weighted/sequential strategies	Elite Set ES + pool IP

Comparison of Advanced Metaheuristic Classes

Key Structural Evolutions:

- **MOTS** → **PSA**: From deterministic tabu memory to stochastic annealing with adaptive weight adjustment.
- **PSA** → **MOPSO**: From trajectory-based cooling to velocity-driven swarm intelligence with hyper-grids.
- **MOPSO** → **2PPLS**: Shift toward parameter-free local search with scalarized initialization.
- **2PPLS** → **MO-GRASP**: Integration of randomized construction strategies with Path Relinking.

Lessons Learned

Key Takeaways

1 Beyond Genetic Algorithms

- Non-GA metaheuristics employ different search mechanisms, including swarm dynamics, tabu memory, local search, and trajectory relinking.

2 Role of Memory and Archives

- External repositories and elite sets retain non-dominated solutions during the search.

3 Parameterization vs. Hyper-Grids

- Adaptive hyper-grids and dynamic weights can help maintain solution diversity.
- 2PPLS provides a parameter-free local-search alternative.

4 Seed Quality Matters

- Supported extreme solutions can provide useful starting points for Pareto local search.

Key References

Bibliography

- Banks, A., Vincent, J., & Anyakoha, C. (2008). A review of particle swarm optimization. Part II: hybridisation, combinatorial, multicriteria and constrained optimization. *Natural Computing*, 7(1), 109–124.
- Coello Coello, C. A., & Lechuga, M. S. (2002). MOPSO: A proposal for multiple objective particle swarm optimization. *Proc. IEEE CEC*, 1051–1056.
- Czyzak, P., & Jaszkiwicz, A. (1998). Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization. *J. Multi-Criteria Decision Analysis*, 7(1), 34–47.
- Gandibleux, X., Mezdaoui, N., & Fréville, A. (1997). A tabu search procedure to solve multiobjective combinatorial optimization problems. *Advances in Multiple Objective and Goal Programming*, 291–300.
- Lust, T., & Teghem, J. (2010). Two-phase Pareto local search for the biobjective traveling salesman problem. *Journal of Heuristics*, 16(3), 475–510.
- Martí, R., Campos, V., Resende, M. G., & Duarte, A. (2015). Multiobjective GRASP with Path Relinking. *European Journal of Operational Research*, 240(1), 54–71.